Lab1

1. Name some common operating systems.
   Windows, MacOS, Android, iOS, Linux
2. Where are these operating systems "operating" on?
   Windows, MacOS, Linux are operating on desktop and notebook computers. Android and iOS are on mobile devices like smartphones and reading pads.
3. Operating systems are software. Do you agree? Give any evidence?
   - Yes, it is a software
   - Operating systems need installation
   - Operating systems is associated with computing devices (desktop, notebook, or mobile)
4. Most people should have experience in using two or more operating systems (OS), for instance, a desktop computer at work and a smartphone for personal matters. Identify and discuss some common functions shared between a mobile OS and a desktop OS.
   Hint: consider from the users' perspective and from the application programs' or programmers' perspective.
   - There are a number of functions shared between operating systems.
   - Some examples from user's perspective:
     - Enables users to run different application programs
     - Enables users to multi-task on the computing device
     - Provides users with an interface for management of application programs and system resources
     - Allows users to interact with application programs
   - Going deeper into the discussion, from the programmers' perspective
     - Allows application programs to obtain memory (i.e., malloc function calls in C or create objects in Java)
     - Allows application programs to read/write file data
     - Allows application programs to read/write data to the Internet

Stage 3: Try the little man computer (LMC).

INP STA NUM INP ADD NUM STA NUM INP ADD NUM STA NUM INP ADD NUM STA NUM OUT HLT NUM DAT	INP STA NUM INP ADD NUM STA NUM INP ADD NUM STA NUM ADD NUM OUT HLT NUM DAT

Lab2

Stage 2: Short questions

1. How could a system be designed to allow a choice of operating systems from which to boot? What would the bootstrap program need to do?
   Consider a system that would like to run both Windows XP and three different distributions of Linux (e.g., RedHat, Debian, and Mandrake). Each operating system will be stored on disk. During system boot-up, a special program (which we will call the boot manager) will determine which operating system to boot into. This means that rather initially booting to an operating system, the boot manager will first run during system startup. It is this boot manager that is responsible for determining which system to boot into. Typically boot managers must be stored at certain locations of the hard disk to be recognized during system startup. Boot managers often provide the user with a selection of systems to boot into; boot managers are also typically designed to boot into a default operating system if no choice is selected by the user.

2. When installing the operating system to the computer, sometimes applications such as web browser and mail programs may come pre-installed with the OS.

   2.1. Could you give some examples?

(a) Microsoft Windows: Internet Explorer and Microsoft Edge are web browsers that come pre-installed with the Windows operating system. For mail, Windows includes a built-in app called Mail, which allows users to manage their email accounts from different providers.
(b) Apple macOS: Safari is the default web browser included with macOS. The operating system also includes a built-in Mail app, which supports a variety of email services.
(c) Linux (Ubuntu): Firefox is the default web browser in many Linux distributions, including Ubuntu. Thunderbird, a mail client also developed by Mozilla, is often included as well.
(d) iOS (Apple's mobile operating system): Safari is the default web browser and the Mail app comes pre-installed on iPhones and iPads.
(e) Android: Google Chrome is the default web browser on most Android devices. Gmail is the default email client, but there's also a generic Email app that can handle other email accounts.

2.2. Consider whether the operating system should include applications such as web browsers and mail programs. Argue both that it should and that it should not, and support your answers.

An argument in favor of including popular applications with the operating system is that if the application is embedded within the operating system, it is likely to be better able to take advantage of features in the kernel and therefore have performance advantages over an application that runs outside of the kernel.
Arguments against embedding applications within the operating system typically dominate however: (1) the applications are applications - and not part of an operating system, (2) any performance benefits of running within the kernel are offset by security vulnerabilities, (3) it leads to a bloated operating system.

Stage 3: Try command line interface (CLI)
Please create a folder call TEST in the Desktop of your computer using CLI.
Answer:
For Windows:
Step 1: Open 'Command Prompt'.
Step 2: Check current directory: *cd*
Step 3: Go to the path of Desktop using: *cd \path_to_Desktop (or cd /d \path_to_Desktop)*
Step 4: Create a folder called TEST: *mkdir TEST*
Step 5: Check whether the folder is created successfully: *dir*
Step 6: Delete the folder: *rmdir TEST*
For MacOS:
Step 1: Open 'Terminal'.
Step 2: Check current directory: *pwd*
Step 3: Go to the path of Desktop using: *cd \path_to_Desktop*
Step 4: Create a folder called TEST: *mkdir TEST*
Step 5: Check whether the folder is created successfully: *ls*
Step 6: Delete the folder: *rmdir TEST*

Lab3

Process	ms using CPU / Burst CPU Utilization Rate 25%	ms doing IO operations IO Utilization Rate 75%
A	4 * 25%= 1	4 × 75% = 3
B	12 * 25%= 3	12 × 75% = 9
C	16 * 25%= 4	16 × 75% = 12

Calculate the CPU utilization rate and the overall turnaround time of the three processes

Total Turnaround Time is 4 + 12 + 16 = 32 ms
Totally the time the CPU is being used is 1 + 3 + 4 = 8 ms
CPU Utilization Rate = (8/32)*100% = 25%

multi-programming scenario,
Total CPU Burst time of 3 process = 1 + 3 + 4 = 8 ms
Total IO burst time of 3 process = 3 + 9 + 12 = 24 ms
The CPU burst time can happen in parallel with the IO burst time, except for the first process.
Turnaround time = 24 ms(IO burst time) + 1 ms (CPU burst time of first process) = 25 ms
CPU Utilization = 8/25 = 32%

Lab4

FCFS	SJF
Non-preemptive Priority based on arrival time Highest priority is the earliest arrived Short processes are disadvantaged in long waiting time and response time	Preemptive or Non-preemptive Priority based on CPU burst time Highest priority is the shortest CPU burst (tie-break: earlier arrival) Long processes may be starved if there are new short processes keep coming in

Processes	Arrival Time (ms)	Estimated CPU Burst (ms)
P1	0	12
P2	4	6
P3	10	2

FCFS

0 --- 12 --- 18 --- 20

|P1 | P2 | P3

	TT	WT	RT
P1	12	0	0
P2	18 – 4=14	8	8
P3	20 – 10=10	8	8
Average	12	5.3	5.3

SJF

0 --- 12 --- 14 --- 20

|P1 | P3 | P2

	TT	WT	RT
P1	12	0	0
P2	20 – 4=16	10	10
P3	14 – 10=4	2	2
Average	10.7	4	4

Preemptive SJF

0 --- 4 --- 10 --- 12 --- 20

|P1 | P2 | P3 | P1

	TT	WT	RT
P1	20	8	0
P2	6	0	0
P3	2	0	0
Average	9.3	2.7	0

The long process P1 that arrived first will hold on to the CPU for nonpreemptive SJF, making shorter processes to wait (i.e. longer waiting time for P2 and P3)

Lab5

Processes	Arrival Time (ms)	Estimated CPU Burst (ms)
P1	0	6
P2	3	5
P3	6	4

In case of a tie, processes with a smaller ID has higher priority.

1. Evaluate the average waiting time, average turnaround time, and average response time if each of the following scheduling algorithm is adopted.

- RR (time quantum = 1ms)
- RR (time quantum = 3ms)

Case1: RR (TQ = 1)

Gantt Chart

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
P1	P1	P1	P1	P2	P1	P2	P1	P3	P2	P3	P2	P3	P2	P3

	TT	WT	RT
P1	8	2	0
P2	14 – 3 = 11	6	4 – 3 = 1
P3	15 – 6 = 9	5	8 – 6 = 2
Average	9.3	4.3	1

		counter	X	X (PCB of A)	X (PCB of B)	Remarks
		10	?	-	-	
A1	LDA X, counter	10	10	-	-	
A2	ADD X, 2	10	12	-	-	
	Context Switching	10	-	12	-	CPU Register X is stored in the PCB of A; then X of B is loaded from PCB to CPU register X
B1	LDA X, counter	10	10	12	-	
B2	SUB X, 2	10	8	12	-	
B3	STO counter, X	8	8	12	-	
	Context Switching	8	12	12	8	CPU Register X is stored in the PCB of B; then X of A is loaded from PCB to CPU register X
A3	STO counter, X	12	12	12	8	

Why Study Operating Systems?

- Fundamental software layer between hardware and applications
- Understanding OS → better understanding of system performance, resource usage, security, concurrency
- Essential for system programming, software development, cybersecurity, cloud computing

What is an Operating System?

- **Definition**: Program that **manages computer hardware**
- Provides base for **application programs**
- Acts as **intermediary** between **users** and **hardware**

Goals of an OS

1. **Convenience** — easier to use computer
2. **Efficiency** — optimal use of resources
3. **Ability to evolve** — (both goals together / adaptability)

Main Functions of OS

- Interface between user and hardware
- Resource allocation (CPU, memory, I/O devices)
- Management of memory, processes, files, security, etc.

Computer System Structure

- **Modern general-purpose system**:
  - 1+ **CPUs**
  - Multiple **device controllers** (disk, USB, video, etc.)
  - Connected via **common bus**
  - Access **shared main memory**
- **Memory controller** synchronizes access to prevent conflicts
- CPU and controllers execute **concurrently**, compete for memory cycles

How an OS Starts

1. **Bootstrap program** (in ROM / firmware)
   - Runs first when power is turned on

- Locates and loads **OS kernel** into memory

2. **Kernel**
   - Core part of OS
   - First OS code loaded
   - **Always resident** in memory
   - Has **direct hardware access**

Evolution of Operating Systems

Type	Convenience	Efficiency	Key Characteristics	Main Problem
OS-less (Serial Processing)	Very low	Low	Manual punch-card loading, one program at a time	Long setup time, CPU idle during I/O
Batch Systems	Low	Medium	Monitor + job sequencing, automatic loading	No user interaction, still one program at a time
Multi-programmed Batch	Medium	High	Multiple programs in memory, CPU switches during I/O	Needs memory protection, scheduling
Time-Sharing Systems	**High**	High	Rapid switching → interactive response	Overhead of context switching

Modern OS → **Time-Sharing Systems** (most convenient + efficient for interactive use)

Problems in Early Systems

OS-less / Serial Processing

- Programmer manually loads programs (punch cards)
- Long setup time → expensive hardware underutilized
- **CPU often idle** during I/O

Batch Systems

- **Monitor** (early OS) automates loading & execution
- Still **one program at a time** → CPU idle during I/O

Uni-programming Limitation

- Processor idle when current program does I/O

Multi-programming Solution

- Keep multiple programs in memory
- When one program waits for I/O → CPU runs another
- → **Higher CPU utilization**

Benefits vs Challenges of Multiprogramming

Benefits

- Higher CPU utilization
- Better throughput
- Some user convenience (multiple jobs)

Challenges

- Need **large memory** to hold multiple programs
- **Memory management** required
- **Memory protection** needed (prevent programs from attacking each other / OS)
- Effective **job scheduling** required

Time-Sharing Systems (Modern Standard)

- Goal: **short response time** for interactive users
- Frequent but short CPU bursts per program
- Users feel computer responds quickly
- Basis of almost all current desktop / mobile / server OS

Protection Mechanisms in Multiprogramming

1. Dual-Mode Operation

- **User mode** — limited privileges (normal apps)
- **Kernel mode** (supervisor / privileged mode) — full access
- **Mode bit** in hardware
- **System calls** → switch to kernel mode temporarily

2. Memory Protection

- Prevent program from accessing another program's memory
- Use **base register** + **limit register**
- OS defines legal memory range for each process

3. CPU Protection (Timer)

- Prevent infinite loops / monopolizing CPU
- OS sets **timer** before giving CPU to user program
- Timer interrupt → returns control to OS periodically

Ch2

Types of Computer Systems

Single Processor Systems

- Simplest setup: One general-purpose processor connected to main memory.
- Processor: Executes instructions.
- Main Memory: Stores programs and data.
- May include special-purpose processors for device-specific tasks (e.g., I/O handling).

Multiprocessor Systems (Parallel or Tightly-Coupled Systems)

- Two or more processors sharing main memory via data buses.
- Executes multiple instructions simultaneously.
- Advantages:
  - Increased throughput (more work done in less time).
  - Economy of scale (cheaper than multiple single-processor systems).
  - Increased reliability (graceful degradation if one processor fails).
- Subtypes:
  - **Asymmetric Multiprocessing**: Master-slave approach; each processor assigned specific tasks.
  - **Symmetric Multiprocessing**: Each processor performs all tasks; more efficient.

Clustered Systems

- Multiple independent systems (nodes) working together, often sharing storage via Storage-Area Network (SAN).
- Provides high-availability (survives failures).
- Subtypes:
  - **Asymmetric Clustering**: One node in hot-standby mode; others run applications.
  - **Symmetric Clustering**: Multiple nodes run applications and monitor each other.
- Some for High-Performance Computing (HPC): Applications must be written for parallelization.

Services Provided by Operating Systems

- OS provides environment for program execution and user-friendly services.
- Key Services:
  - **User Interface (UI)**:
    - Command-Line Interface (CLI).
    - Graphical User Interface (GUI): Uses mouse, keyboard, monitor; desktop metaphor.
    - Touch-Screen: Gestures, virtual keyboard (no mouse needed).
  - **Program Execution**: Load program into memory, run it, handle normal/abnormal termination.
  - **I/O Operations**: Manage input/output for files or devices.
  - **File-System Manipulation**: Read/write files/directories, create/delete, search, list info, permissions.
  - **Communications**: Exchange info between processes (same computer or networked); via shared memory or message passing.
  - **Error Detection**: Monitor for errors in CPU, memory, I/O, programs; take corrective actions; provide debugging tools.
  - **Resource Allocation**: Allocate resources (CPU cycles, memory, storage, I/O) to multiple users/jobs.
  - **Accounting**: Track resource usage by users.
  - **Protection and Security**: Control access to resources; user authentication; prevent interference between processes; defend against external threats.

System Calls and System Programs

System Calls

- Fundamental interface between user programs and kernel.
- Controlled entry to kernel for privileged services (e.g., I/O, process control, memory management).
- Programs typically use APIs (e.g., C library) that wrap system calls, not direct calls.
- Example Sequence (Copy File Contents):
  1. Acquire input file name (prompt, accept input).
  2. Acquire output file name (prompt, accept input).
  3. Open input file (abort if doesn't exist).
  4. Create output file (abort if exists).
  5. Loop: Read from input, write to output until end.
  6. Close output file.

7. Write completion message.
8. Terminate normally.

Examples of Windows and Unix System Calls

Category	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communications	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Why Prefer APIs Over Direct System Calls?

- **Program Portability**: Code compiles/runs on any system supporting the API.
- **Simplified Programming**: APIs are easier than complex system calls.
- **Correlation with System Calls**: APIs map closely to kernel calls.
- **Similarity to Native Calls**: POSIX/Windows APIs align with Unix/Linux/Windows native calls.
- Advantages of APIs:
  - Abstraction: Hide complexity; use simple functions.
  - Portability: Standardized (e.g., POSIX, Windows API).
  - Implementation: Wrappers that invoke system calls.
  - Standards: Match major OS native calls.

Dual-Mode Protection

- Two modes for safety:
  - **User Mode**: Limited privileges; user programs run here; access only own memory; no direct hardware/privileged instructions; use system calls for kernel services.
  - **Kernel Mode**: Privileged; executes harmful instructions; hardware bit indicates mode.
- System calls switch to kernel mode for privileged operations.
- Protects hardware and prevents attacks.

Types of System Calls

- **Process Control**:
  - Create/terminate process, end/abort.
  - Load/execute.
  - Get/set attributes.
  - Wait for time/event, signal event.
  - Allocate/free memory.
- **File Management**:
  - Create/delete file.
  - Open/close.
  - Read/write/reposition.
  - Get/set attributes.
- **Device Management**:
  - Request/release device.
  - Read/write/reposition.
  - Get/set attributes.
  - Attach/detach devices.
- **Information Maintenance**:
  - Get/set time/date.
  - Get/set system data.
  - Get/set process/file/device attributes.
- **Communications**:
  - Create/delete connection.
  - Send/receive messages (client-server).
  - Transfer status.
  - Attach/detach remote devices.

System Programs

- Provide convenient environment for program development/execution.
- OS is a program managing hardware, basis for applications, intermediary between user and hardware.
- Categories:
  - **File Management**: Create/delete/copy/rename/print/dump/list/manipulate files/directories.
  - **Status Information**: Query date/time/memory/disk/users; performance/logging/debugging; format/print output.
  - **File Modification**: Text editors; search/transform text.
  - **Programming-Language Support**: Compilers/assemblers/debuggers/interpreters (e.g., C, C++, Java).
  - **Program Loading and Execution**: Loaders (absolute/relocatable), linkage editors, overlay-loaders, debuggers.
  - **Communications**: Virtual connections; messaging, web browsing, email, remote login, file transfer.

System Programs vs. Application Programs

- **Application Programs**: User tools for work (e.g., word processor, browser).
- **System Programs**: Manage computer/support applications (e.g., file manager, shell, device drivers).

CH3

Process Concept

- **Process**: A program in execution. An OS executes various programs as processes.
- **From Programs to Processes**:
  - A program is a static set of instructions on disk.
  - OS creates processes when it receives a command to execute a program.
  - One program can be executed multiple times to create multiple processes.
  - Each process is an abstract entity modeling the state of a running program and has its own resource allocation (e.g., memory, CPU).
- **Resource Needs of a Program**:
  - Example C code: Function to sum an array and main function reading from file "input.txt".
  - Resources involved: CPU (for computation), Memory (for array/storage), Hard Disk (for file I/O).
- **Process State**:
  - As a process executes, it changes state.

- States:
  - **New**: Process is being created.
  - **Running**: Instructions are being executed.
  - **Waiting**: Process waiting for an event (e.g., I/O).
  - **Ready**: Process waiting to be assigned to a processor.
  - **Terminated**: Process has finished execution.
- Diagram: New → Ready → Running → Waiting (loops via I/O/event), Running → Terminated (exit), Running → Ready (interrupt/scheduler dispatch).

- **Life Cycle of a Process** (Diagram Summary):
  - Loading OS: CPU executes OS from hard disk to main memory.
  - OS Started: OS receives command to execute a program.
  - Process Creation: OS loads program into memory and creates process.
  - Process Ready: Process in main memory, ready for execution; OS decides to pass CPU to the program.
  - Process Running: Process controls CPU, being executed.
  - Process Waiting: Process calls OS to do I/O (e.g., input/output); OS may control CPU normally until I/O ends.
  - Process Interrupted: Interrupt occurs (e.g., timer); control passed back to OS via Interrupt Service Routine. Process now ready for re-execution.
  - Process Terminating: Program finished execution or terminated; control passed to OS.
  - Process Terminated: OS releases resources associated with the process; memory space returned to OS; program terminates itself or is terminated.

Process Control Block (PCB)

- Also called Task Control Block.
- Represents each process in the OS.
- Contains:
  - Process ID/Number
  - Process State (running, waiting, etc.)
  - Program Counter (next instruction location)
  - CPU Registers (process-centric registers)
  - CPU Scheduling Info (priorities, queue pointers)
  - Memory Management Info (allocated memory)
  - Accounting Info (CPU used, elapsed time, limits)
  - I/O Status Info (allocated devices, open files)
  - Memory Limits
  - List of Open Files

- Diagram: Table-like structure listing these fields.

Process Scheduling

- Objectives:
  - Multiprogramming: Keep some process running to maximize CPU utilization.
  - Time Sharing: Switch CPU frequently for user interaction.
- Process Scheduler: Selects available process for execution on CPU.
- In single-processor system: Only one running process; others wait.
- Queues:
  - Ready Queue: Processes in main memory, ready to execute.
  - Wait Queues: Processes waiting for events (e.g., I/O).
- Processes migrate between queues.
- Responsibilities: Decide which process executes, which waits, handle priorities.

Context Switch

- Occurs when CPU switches from one process to another.
- Triggered by interrupts (OS changes CPU task to kernel routine).
- Analogy: Reading a book, interrupted by mom (higher priority); use bookmark (save state) to resume.
- Steps:
  - Save state of old process (into PCB).
  - Load saved state of new process (from PCB).
- Context-switch time is overhead (no useful work); depends on hardware/OS complexity.
- More complex PCB = longer switch.
- Hardware support (e.g., multiple register sets) reduces time.

Process Operations (Creation & Termination)

- OS provides mechanisms for creation and termination.
- **Creation**:
  - Process creates children via system call (e.g., fork() in UNIX).
  - Forms tree: Parent → Children.
  - Identified by PID (Process Identifier).
  - Address Space: Child duplicates parent or loads new program.
  - Execution: Parent/children concurrent, or parent waits (e.g., wait()).
  - UNIX: fork() creates copy; exec() replaces memory with new program.
- **Termination**:

    - Process calls exit() after last statement; returns data to parent (via wait()).
    - Resources deallocated (memory, files, buffers).
    - Parent can abort child: Exceeded resources, task obsolete, parent exiting (cascading termination).

Multithreading

- **Problem with Single Thread**: Poor user experience; one thread per process means can't multitask (e.g., respond while working).
- **Thread**: Lightweight process within a process.
    - Shares process resources (code, data, files).
    - Has own: Program Counter, registers, stack, state.
- **Single vs. Multithreaded**:
    - Single: One thread.
    - Multi: Multiple threads sharing resources.
- **Benefits**:
    - Responsiveness: Continue if part blocked (e.g., UI).
    - Resource Sharing: Easier than IPC.
    - Economy: Cheaper creation/switching than processes.
    - Scalability: Utilizes multicore; single-thread limited to one core.

Program vs. Process vs. Thread

Concept	Description	Example in Microsoft Word
Program	Static instructions on disk.	.exe file of Word. Opening creates process; multiple openings = multiple processes.
Process	Instance of program in execution.	Word instance with its memory/resources.
Thread	Lightweight execution unit within process.	Threads for typing, spell-checking, auto-saving; share document memory but concurrent tasks.

Interprocess Communication (IPC)

- Processes: Independent (no sharing/affecting) or Cooperating (share data/affect each other).
- Reasons for Cooperation: Info sharing, speedup, modularity, convenience.
- Models: Shared Memory or Message Passing.

IPC in Shared-Memory Systems

- Shared memory area among processes.
- Resides in creator's address space; others attach.
- OS normally prevents access; processes agree to share.
- Processes read/write directly.

IPC in Message-Passing Systems

- Communicate without shared address space (useful for distributed systems).
- Operations: send(message), receive(message).
- Message Size: Fixed (simple implementation, hard programming) or Variable (complex implementation, easy programming).

Key Differences Between Shared Memory and Message Passing

Aspect	Shared Memory	Message Passing
Communication Method	Read/write to shared memory.	Send/receive via channel.
Speed	Faster (no copying).	Slower (data copied).
Synchronization	Explicit (e.g., mutexes).	System-handled.
Complexity	More complex.	Easier.
Scalability	Same machine only.	Distributed systems.
Use Case	High-performance, large data.	Simplicity, distributed.

CH4

Basic Concepts

- **Maximum CPU Utilization**: Achieved through multiprogramming (running multiple processes).
- **CPU-I/O Burst Cycle**: Process execution alternates between CPU bursts (computation) and I/O bursts (waiting for I/O).
- CPU burst followed by I/O burst.
- Example cycle: Load/store/add from file → CPU burst → Wait for I/O → I/O burst → Repeat.
- **CPU Burst Distribution**: Main concern for scheduling; histograms show a large number of short CPU bursts and a small number of long bursts.
- **Histogram Insight**: Frequency high for short burst durations, decreases as duration increases.

CPU Scheduler

- Selects a process from the ready queue and allocates a CPU core to it.
- Queue can be ordered in various ways (e.g., FIFO, priority).
- Scheduling decisions occur when a process:

1. Switches from running to waiting state (e.g., I/O request).
2. Switches from running to ready state (e.g., interrupt).
3. Switches from waiting to ready state (e.g., I/O completion).
4. Terminates.

- For cases 1 and 4: No choice; select next ready process if available.
- For cases 2 and 3: Choice exists (preemptive scheduling possible).

Diagram of Process States

- New → Admitted → Ready
- Ready → Scheduler dispatch → Running
- Running → Interrupt → Ready
- Running → Interrupt → Ready
- Running → I/O or event wait → Waiting
- Waiting → I/O or event completion → Ready
- Running → Exit → Terminated

Preemptive vs. Nonpreemptive Scheduling

- **Nonpreemptive**: Scheduling only in cases 1 and 4; process keeps CPU until it terminates or enters waiting state.
- **Preemptive**: Scheduling in all cases (including 2 and 3); can interrupt running process.
- Modern OS (Windows, MacOS, Linux, UNIX) use preemptive scheduling.

Dispatcher

- Gives CPU control to the selected process.
- Switching context (save/restore state in PCB).
- Switching to user mode.
- Jumping to the proper location in the user program to restart.
- **Dispatch Latency**: Time to stop one process and start another (typically small, <10 microseconds).

Scheduling Criteria

- **CPU Utilization**: Keep CPU as busy as possible (maximize % busy time).
- **Throughput**: Number of processes completed per unit time (maximize).
- **Turnaround Time**: Time from process submission to completion (minimize).
- **Waiting Time**: Total time spent in ready queue (minimize).
- **Response Time**: Time from request submission to first response (minimize; important for interactive systems).

Scheduling Algorithm Optimization Criteria

- Maximize CPU utilization.
- Maximize throughput.
- Minimize turnaround time.
- Minimize waiting time.
- Minimize response time.

Various Scheduling Algorithms

First-Come, First-Served (FCFS)

- Simplest algorithm; nonpreemptive by default.
- Processes requesting CPU first get it (FIFO queue).
- Process at head of queue gets CPU when free.
- **Convoy Effect**: Short processes wait behind long ones, increasing average waiting time.
- **Problems**: Troublesome for time-sharing; long processes can monopolize CPU.
- **Practice Problem** (With Arrivals; Nonpreemptive; Tie breaker: Smaller ID higher priority):

Shortest-Job-First (SJF)

- Associates priority with next CPU burst length (shorter = higher priority).
- Schedules process with smallest next CPU burst.
- Can be preemptive (SRTF: Shortest Remaining Time First) or nonpreemptive.
- **Challenge**: Predicting next burst length (often use exponential averaging of past bursts).
- **Nonpreemptive** :
- **Preemptive Example** (With Arrivals):

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9

Process	Arrival Time	Burst Time
P4	3	5

- Gantt: P1 (0-1), P2 (1-5), P4 (5-10), P3 (10-17), P1 (17-24)

Priority Scheduling

- Each process has a priority (integer; smaller = higher).
- CPU to highest priority process.
- Can be preemptive or nonpreemptive.
- SJF is a special case (priority = 1/burst).
- **Problem**: Starvation (low-priority processes may never run).
- **Solution**: Aging (gradually increase priority of waiting processes, e.g., +1 every 15 min).

Round-Robin (RR)

- Designed for time-sharing; preemptive.
- Time quantum (slice): 10-100 ms.
- Ready queue as FIFO; run process up to quantum, then move to tail if unfinished.
- If burst < quantum, process releases CPU voluntarily.
- Higher turnaround than SJF, but better response time.
- **Time Quantum Trade-offs**:
- Too long: Degenerates to FCFS.
- Too short: Excessive context switches (overhead).
- Quantum >> context switch time (e.g., quantum 10-100 ms, switch <10 μs).

Multilevel Queue

- Multiple ready queues.
- Parameters:
- Number of queues.
- Scheduling algorithm per queue (e.g., RR in one, FCFS in another).
- Queue assignment for processes (e.g., by type: system, interactive, batch).
- Scheduling among queues (e.g., fixed priority, time slices).
- Example: Separate queues by priority (0 highest); schedule highest queue first.
- Prioritization by process type (e.g., foreground interactive vs. background batch).

Priority Scheduling with Round-Robin

- Highest priority queue runs first; within same priority, use RR.

Multilevel Feedback Queue

- Processes can move between queues (dynamic).
- Allows aging to prevent starvation.
- Parameters:
- Number of queues.
- Algorithm per queue.
- Upgrade method (e.g., after waiting time).
- Demote method (e.g., after using quantum).
- Entry queue for new processes.
- Example: Higher queues for short bursts (RR small quantum); lower for long (FCFS).

(Complete-Arrival) | (TT- Burst) | (1st run -Arrival)